Machine Learning–Based Late Delivery Risk Prediction in Global Supply Chain Operations

Prepared By: Trisha Dutta

1. Introduction

In global supply chain operations, late deliveries lead to penalties, customer dissatisfaction, and operational inefficiencies. This project aims to build a machine learning model to predict whether an order will be delivered late before shipment. This enables proactive decision-making and better logistics planning.

2. Dataset Overview

The dataset consists of 180,000+ records with information related to orders, customers, shipping details, and product attributes. The target variable is **Late_delivery_risk**, which indicates whether a delivery was late (1) or not (0).

```
4    print(df.head())
```

Sample of dataset

3. Data Preprocessing

Irrelevant columns such as customer name, address, and product details were removed.
Categorical variables were converted into numerical format using one-hot encoding.
The dataset was checked for missing values, and no missing values were found.

```
6    print(df.isnull().sum())
```

Missing value check

4. Feature Engineering

New features were created to improve model performance:

- **Delay Gap** = Difference between actual and scheduled shipping days
- **Order Complexity** = Quantity × Product Price

These features help capture operational delays and order difficulty.

```
14    df["Delay_Gap"] = df["Days for shipping (real)"] - df["Days for shipment (scheduled)"]
15
16    df["Order_Complexity"] = df["Order Item Quantity"] * df["Order Item Product Price"]
```

Feature engineering

5. Model Building

The dataset was split into training and testing sets (80-20).
Data scaling was applied using StandardScaler.
Two models were trained:

- Logistic Regression (baseline)
- Random Forest (final model)

6. Model Performance

```
Accuracy: 0.9668734766230889
              precision    recall  f1-score   support

           0       0.98      0.95      0.96     16384
           1       0.96      0.98      0.97     19720

    accuracy                           0.97     36104
   macro avg       0.97      0.97      0.97     36104
weighted avg       0.97      0.97      0.97     36104

['Low Risk', 'Low Risk', 'Low Risk', 'High Risk', 'Low Risk', 'Low Risk', 'High Risk', 'Low Risk', 'High Risk', 'Low Risk']
ROC-AUC: 0.9984593188303238
Random Forest Accuracy: 0.9782295590516287
              precision    recall  f1-score   support

           0       1.00      0.95      0.98     16384
           1       0.96      1.00      0.98     19720

    accuracy                           0.98     36104
   macro avg       0.98      0.98      0.98     36104
weighted avg       0.98      0.98      0.98     36104
```

Model performance

Logistic Regression Accuracy: 96.68%
Random Forest Accuracy: 97.82%
ROC-AUC Score: 0.998

The Random Forest model showed better performance and was selected as the final model.

7. Risk Prediction

The model assigns a probability score to each order and categorizes it into:

- Low Risk (<0.3)
- Medium Risk (0.3–0.7)
- High Risk (>0.7)

```
['Low Risk', 'Low Risk', 'Low Risk', 'High Risk', 'Low Risk', 'Low Risk', 'High Risk', 'Low Risk', 'High Risk', 'Low Risk']
```

Risk categorization output

8. Key Insights

Feature importance analysis shows that the most critical factors affecting delays are:

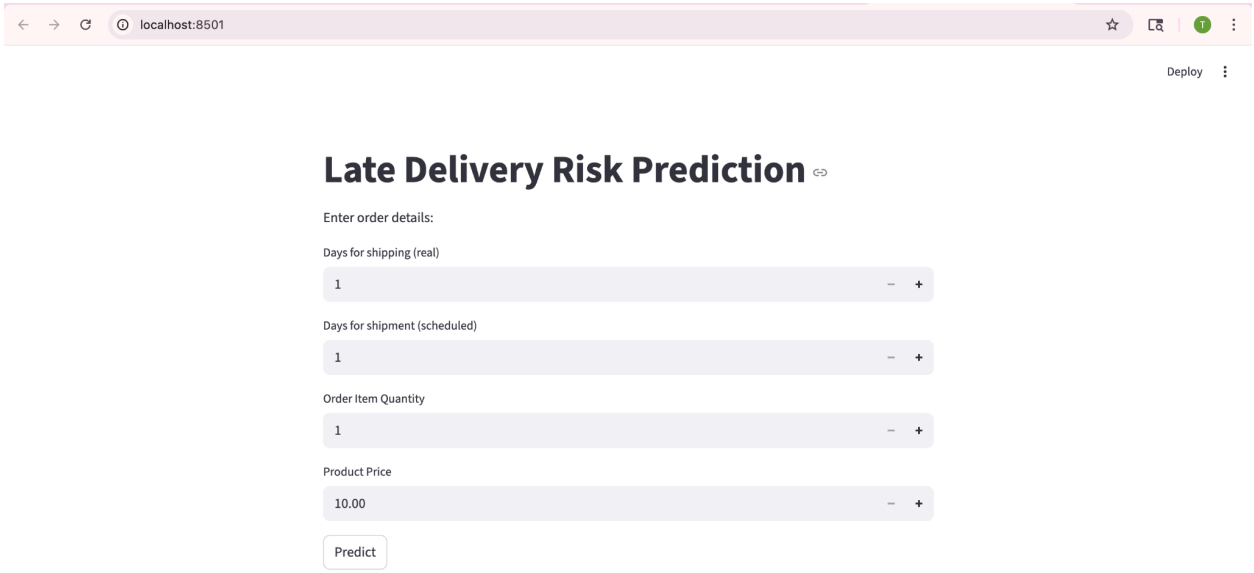
- Delay Gap
- Shipping duration
- Shipping mode

```
Top 5 Important Features:
Delay_Gap                        0.395442
Days for shipping (real)         0.274083
Days for shipment (scheduled)    0.083049
Shipping Mode_Standard Class     0.082725
Shipping Mode_Second Class       0.017621
dtype: float64
```

Feature importance

9. Streamlit Dashboard

A simple Streamlit dashboard was developed to predict late delivery risk. Users can input order details such as shipping days, quantity, and price to get real-time risk prediction.



Streamlit Dashboard Interface

The application was executed locally using the Streamlit framework, and allows users to input order details to obtain real-time risk predictions.

The Streamlit application was developed and executed locally using the command: streamlit run app.py. The dashboard allows users to input order details and obtain real-time risk predictions.

10. Conclusion

The project successfully predicts late delivery risk using machine learning. It helps identify high-risk orders in advance, enabling better planning, reducing delays, and improving customer satisfaction.

Appendix: Python Implementation

1. Machine Learning Model Code (UM.py)

```
import pandas as pd

import numpy as np

df = pd.read_excel("/Users/saibesh/Desktop/Study Material/UM/APL_Logistics.xlsx")

print(df.head())

print(df.info())

print(df.isnull().sum())

df = df.drop([

   "Customer Fname", "Customer Lname", "Customer Street",

   "Product Name", "Customer City", "Order City",

   "Customer Country", "Order Country",

   "Delivery Status", "Order Status"

], axis=1)



df["Delay_Gap"] = df["Days for shipping (real)"] - df["Days for shipment (scheduled)"]



df["Order_Complexity"] = df["Order Item Quantity"] * df["Order Item Product Price"]



df = pd.get_dummies(df, drop_first=True)

X = df.drop("Late_delivery_risk", axis=1)

y = df["Late_delivery_risk"]
```

```
from sklearn.model_selection import train_test_split



X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)

from sklearn.preprocessing import StandardScaler



scaler = StandardScaler()



X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

from sklearn.linear_model import LogisticRegression



model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score



print("Accuracy:", accuracy_score(y_test, y_pred))

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))

print("Accuracy:", accuracy_score(y_test, y_pred))



from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))




# 👇 ADD THIS WHOLE BLOCK BELOW


from sklearn.ensemble import RandomForestClassifier


rf = RandomForestClassifier(n_estimators=100, random_state=42)

rf.fit(X_train, y_train)


y_pred_rf = rf.predict(X_test)

from sklearn.metrics import roc_auc_score


y_prob = rf.predict_proba(X_test)[:,1]

def risk_category(p):

   if p < 0.3:

       return "Low Risk"

   elif p < 0.7:

       return "Medium Risk"

   else:

       return "High Risk"


risk_labels = [risk_category(p) for p in y_prob]
```

```
print(risk_labels[:10])  # just to see first 10 results



print("ROC-AUC:", roc_auc_score(y_test, y_prob))



from sklearn.metrics import accuracy_score, classification_report



print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

print(classification_report(y_test, y_pred_rf))

import pandas as pd



importance = pd.Series(rf.feature_importances_, index=X.columns)

print("Top 5 Important Features:")

print(importance.sort_values(ascending=False).head(5))
```

2. Streamlit Application Code (app.py)

```
import streamlit as st
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Load data
df = pd.read_excel("/Users/saibesh/Desktop/Study Material/UM/APL_Logistics.xlsx")

# Drop columns
df = df.drop([
    "Customer Fname", "Customer Lname", "Customer Street",
    "Product Name", "Customer City", "Order City",
    "Customer Country", "Order Country",
    "Delivery Status", "Order Status"
], axis=1)

# Feature Engineering
df["Delay_Gap"] = df["Days for shipping (real)"] - df["Days for shipment (scheduled)"]
df["Order_Complexity"] = df["Order Item Quantity"] * df["Order Item Product Price"]

# Encoding
df = pd.get_dummies(df, drop_first=True)

# Split
X = df.drop("Late_delivery_risk", axis=1)
y = df["Late_delivery_risk"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
rf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf.fit(X_train, y_train)

# UI
st.title("Late Delivery Risk Prediction")

st.write("Enter order details:")

shipping_real = st.number_input("Days for shipping (real)", 1, 10)
shipping_scheduled = st.number_input("Days for shipment (scheduled)", 1, 10)
quantity = st.number_input("Order Item Quantity", 1, 10)
price = st.number_input("Product Price", 10.0, 500.0)

# Feature creation
delay_gap = shipping_real - shipping_scheduled
order_complexity = quantity * price

# Dummy input (simplified)
input_data = np.zeros(X.shape[1])

# Put values (basic approximation)
input_data[0] = shipping_real
input_data[1] = shipping_scheduled
input_data[2] = quantity
input_data[3] = price

# Predict
if st.button("Predict"):
   input_scaled = scaler.transform([input_data])
   prob = rf.predict_proba(input_scaled)[0][1]

   if prob < 0.3:
       risk = "Low Risk"
   elif prob < 0.7:
       risk = "Medium Risk"
   else:
       risk = "High Risk"

   st.write(f"Risk Probability: {prob:.2f}")
   st.write(f"Risk Category: {risk}")
```